AMATH 482/582: HOME WORK 2

VARUN PANUGANTI

ACMS Department, University of Washington, Seattle, WA
varunp5@uw.edu

Abstract. In this report, I us PCA truncation to build a lower dimension projection of a robots movements categorized into 3 different categories (running, jumping, and walking) with samples provided for each category. I did this by first compiling all samples into a single matrix, and observing the cumulative explained variance (what number of modes/components are most efficient at capturing the original information). I then categorized them into their movement categories, applied PCA truncation, and visualized them in the 2D and 3D plane. I then built a classifier algorithm, by computing the centroid for each movement category and mapping each trajectory to each centroid. Finally I computed the train and test accuracy for each k-mode and determined which mode was more efficient.

1. Introduction and Overview

In this report I was working on the next version of a humanoid robot OptimuS-VD which has built-in sensors that record the movements of 38 of its joints with rate of 60Hz. The movements of the joints are recorded as Euler angles and can be transformed to xyz coordinates. I have 5 recorded samples of each of the 3 movements that OptimuS-VD knows to perform: walking, jumping, and running. The samples are recorded for 1.4 secs (100 timesteps) and saved as a matrix of 114×100, where the first dimension records $x_1, .., x_{38}, y_1, .., y_{38}, z_1, .., z_{38}$ locations of the joints and the second dimension the timesteps. In this report, I built a projection of the recordings to a lower dimension than the number of coordinates and created visualizations of the movements. Then I designed a classification algorithm by calculating the centroids of each movement using the truth labels and then mapped each sample to each movement based on closest distance.

2. Theoretical Background

2.1. **PCA Setup.** First we begin by creating $X \in R^{114*N}$ Before applying PCA we must center the data: $\mu = \frac{1}{N}\sum_{i=1}^{N} X_i$. While this next part was done using the PCA Python library, the math looks like: 1. Compute the SVD of X 2. Find the PC modes. Let $U^k = [U_1, ..., U^K]$ which are the top-k left singular vectors.

After finding the PC modes I obtained the PCA variance ratio: $1 = \sum_{i=1}^{k} \lambda_i$. I then investigated how much of the data is preserved if I keep the first k modes. Then, I computed the PCA coordinates using the formula $(U^k)^T(X - \mu)$ for $k = 2, 3$ and where $(X - \mu$ are the centered points.

2.2. **Classification.** Using the truth labels provided, I computed the centroid (mean) in k-modes PCA space for each movement: 1. Pick the category (running, jumping, walking) 2. Compute the mean PCA coordinate over time: $mPc_i^k = \frac{1}{T}\sum_{j=1}^{T} PCA_i^k(:, j)$ where mPc_i^k is the mean PCA coordinate in the kth dimension, i corresponds to the ith projected trajectory, and $PCA_i^k(:, j)$ is the coordinate of the ith projected trajectory at the jth time. 3. Compute the centroid for each

Date: March 18, 2026.

movement category: $C_m^k = \frac{1}{n}\sum_{i=1}^{n} mPc_i^k$ where m corresponds to each movement category and C_m^k is the centroid of that movement category in the kth dimension.

Finally we have C_m^k for m in "running", "jumping", "walking" and k in 2,3. Now we can do our predictions: 1. $point_i^k$ = centered data converted to PCA coordinates in k^{th} dimension. Same theory as subsection 2.1 "PCA Setup". 2. chosen_centroid = $\arg\min_{\ell\in\{0,1,2\}} ||point_i^k - centroid_i^k||_2$ We run this procedure for all points in i = 1,...,number of trajectories and k = 2,3.

Finally, we compute the accuracy for all dimensions in k = 2,3: $Accuracy_k = \frac{1}{N}(label_i = predicted_label_i^k)$ where $label_i$ is the ith truth label, and $predicted_label_i^k$ is the ith predicted label using the k-mode PCA.

3. Algorithm Implementation and Development

In my code I used the library PCA from sklearn to help compute the PCA of my data, and calculate the cumulative energy. I used accuracy_score from sklearn.metrics to compute the accuracies of my classifications [4]. I also used Matplotlib [2] and go (from plotly) [3] to create the 2D and 3D visualizations of the trajectories. I used numpy [1] (provided in the starter notebook) for operations like computing the mean (for the purpose of centering), computing the distance between points (for the purpose of choosing centroids), and computing the cumulative explained variance.

4. Computational Results

Here I will be displaying the visualizations and findings from my code based on the theoretical background above.

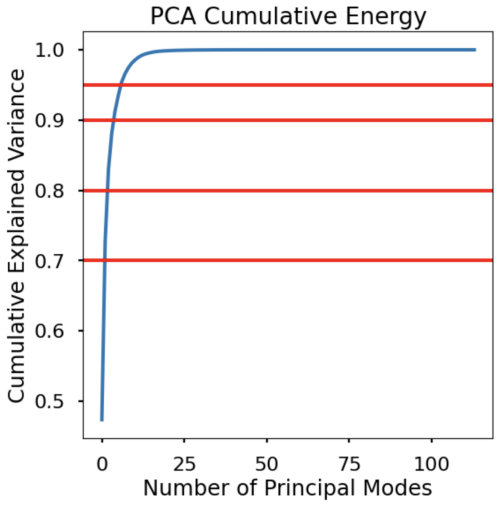

Figure 1. Here you can see the relationship between the number of PCA modes and cumulative explained variance. There seems to be an asymptote right around Cumulative Explained Variance = 1, such that any number of principal modes beyond 10 does not do much better in terms of explaining and preserving the original data.

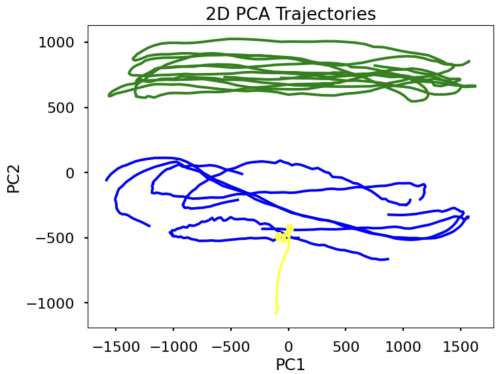

FIGURE 2. [2] These are the PCA trajectories displayed in 2D. Blue is walking, green is running, yellow is jumping. You can see that the trajectories of the movement categories are cleanly separated. Also, running and walking form repetitive loops while jumping follows a trajectory that is up and down, meaning it is not as periodic.

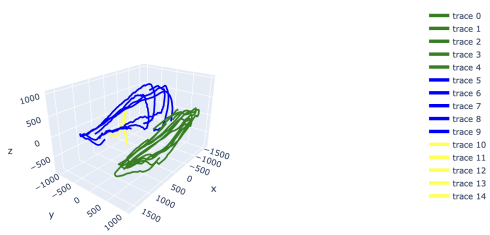

FIGURE 3. [3] These are the PCA trajectories displayed in 3D. Blue is walking, green is running, yellow is jumping. When observing closer you can see that the trajectories look very similar to the 2D trajectories, with the jumping trajectory going straight up while the running and walking trajectories are going back and forth in repetitive loops.

k	train accuracy	test accuracy
2	0.933	1.0
3	0.8	1.0

TABLE 1. [4] This is one of the most important visualizations when it comes to analyzing the effect of different truncation levels. It shows the computed classification accuracy on both the train and test data, and for each k-mode. One thing that can be pointed out straight away is a 100% test accuracy for both k-modes. This is probably because we are only working with 3 motion classes and PCA is very efficient at pattern recognition. However, in the lower k mode of 2, we have a higher train accuracy of 0.933 than the higher k mode of 3 which gives us a lower train accuracy of 0.8. This shows that adding additional PCA modes does not improve our performance and could even be more computationally expensive.

5. Summary and Conclusions

Overall, I can summarize that these results show that PCA provides an efficient low-dimensional representation of the robots motion. My perspective is that even with a lower number of principal

modes/components, we can still preserve the data. This is backed up by figure 1 showing the relationship between the number of principal modes and cumulative explained variance. You can see that less than k=10 principal modes are necessary to explain 90% of the data. Furthermore table 1 also gives an insight showing how k=2 beats k=3 in train accuracy and ties it in test accuracy. I also think that from a qualitative perspective I hold a similar opinion. Looking at the 2D vs 3D trajectories, the 2D trajectories tell almost the same story. Ultimately, I think that using PCA truncation is efficient and computationally inexpensive.

Acknowledgements

The author is thankful to Prof. Frank for useful discussions about how to compute PCA truncation.

References

[1] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, Robert Kern, Matti Picus, Stephan Hoyer, Marten H. van Kerkwijk, Matthew Brett, Allan Haldane, Jaime Fernández del Río, Mark Wiebe, Pearu Peterson, Pierre Gérard-Marchant, Kevin Sheppard, Tyler Reddy, Warren Weckesser, Hameer Abbasi, Christoph Gohlke, and Travis E. Oliphant. Array programming with NumPy. *Nature*, 585(7825):357–362, September 2020.

[2] J. D. Hunter. Matplotlib: A 2d graphics environment. *Computing in Science & Engineering*, 9(3):90–95, 2007.

[3] Plotly Technologies Inc. Collaborative data science, 2015.

[4] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.